

Static Analysis for LLM Code Repair: A Methodology Study

Anonymous Authors¹

Abstract

Iterative LLM code repair relies primarily on execution feedback—test results and stack traces. Static analysis offers complementary signal: mechanistic explanations of why code is problematic, not just where it fails. Prior work demonstrates that static analysis improves code quality metrics (security, readability), but the impact on functional correctness (pass@k) remains unmeasured.

This paper presents a methodology study for evaluating static analysis in LLM code repair. A gate-based hypothesis validation framework is developed alongside a pylint analysis pipeline for HumanEval. The existence gate tests whether static analyzers produce actionable warnings on benchmark code—a prerequisite for any intervention to have signal.

The primary finding is that canonical HumanEval solutions exhibit only 9.15% pylint warning rate (15 of 164 problems with at least one warning, 23 total warnings across 9 categories). This result makes canonical solutions unsuitable proxies for LLM-generated code, which typically contains more issues. The gate correctly halted downstream experiments before resources were wasted on invalid methodology.

Contributions include: (1) the first baseline measurement of static warning rates on HumanEval canonical solutions; (2) a validated pylint pipeline for code generation research; (3) a gate-based validation methodology that catches methodology limitations early; and (4) identification of the canonical-vs-LLM proxy limitation. The hypothesis that static analysis improves pass@k remains untested.

The validated infrastructure and baseline are provided for future work with LLM API access.

1. Introduction

Large language models have transformed code generation, achieving strong performance on benchmarks such as HumanEval and MBPP. Iterative repair loops—where models refine their output based on feedback—have emerged as a key technique for improving functional correctness. The dominant paradigm, exemplified by Self-Debug (Chen et al., 2023), relies on execution feedback: test results and stack traces guide the model toward working solutions.

Static analysis offers an alternative signal. Tools such as pylint and mypy detect potential issues—undefined variables, type mismatches, unreachable code—before execution. Unlike execution traces that show where code fails, static warnings explain why the code is problematic. This mechanistic feedback could enable more targeted repairs.

The research gap. Prior work has shown that static analysis improves code quality metrics. Blyth et al. (2025) demonstrated reductions in security issues (40% to 13%) and readability problems (80% to 11%) when integrating static feedback into LLM pipelines. However, no published study has measured the impact on functional correctness—the pass@k metrics that matter for code generation benchmarks.

Research question. Does integrating static analyzer feedback into iterative LLM code repair improve pass@k compared to execution-only feedback?

This work. This paper presents a methodology study that establishes groundwork for answering this question. A hypothesis validation framework with gate-based testing is developed, a pylint analysis pipeline for HumanEval is implemented, and baseline measurements are provided. The existence gate—testing whether static analyzers produce actionable warnings on benchmark code—reveals a methodological limitation: canonical solutions exhibit only 9.15% warn-

¹Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

ing rates, making them unsuitable proxies for LLM-generated code.

Contributions:

1. First baseline measurement of static warning rates on HumanEval canonical solutions (9.15%, 23 warnings across 9 categories)
2. Validated pylint analysis pipeline for code generation research with reusable components
3. Gate-based hypothesis validation methodology that catches methodology limitations before full experiments
4. Identification of proxy limitation: canonical solutions do not represent LLM output for static analysis evaluation

This work is framed as a methodology contribution. The hypothesis that static analysis improves pass@k remains untested, but the validated pipeline and baseline are provided for future work with LLM API access.

2. Related Work

2.0.1. 2.1 ITERATIVE CODE REPAIR

Self-Debug (Chen et al., 2023) introduced execution-feedback loops for LLM code generation, using test results and stack traces to guide iterative refinement. Self-Refine (Madaan et al., 2023) generalized this to broader refinement tasks using LLM self-feedback. These methods establish execution feedback as the baseline for repair loops.

LDB (Zhong et al., 2024) extended this paradigm with block-by-block verification, improving localization of errors. However, these approaches rely on execution traces—they identify where code fails but not why.

2.0.2. 2.2 STATIC ANALYSIS FOR LLMs

Blyth et al. (2025) represent the closest prior work. They integrated static analysis feedback (pylint, bandit) into LLM pipelines and demonstrated improvements in code quality metrics: security issues reduced from 40% to 13%, and readability problems reduced from 80% to 11%. Notably, they did not measure functional correctness (pass@k). Their contribution establishes that LLMs can interpret static warnings, but whether this improves benchmark performance remains unknown.

Chen et al. (2024) proposed combining testing and static analysis feedback for code generation, though

their evaluation focused on quality metrics rather than pass@k.

2.0.3. 2.3 THE GAP ADDRESSED

Prior Work	Feedback Type	Metric	Pass@k Measured
Self-Debug (Chen et al., 2023)	Execution	Functional correctness	Yes
Self-Refine (Madaan et al., 2023)	LLM self-feedback	Task completion	Yes
Blyth et al. (2025)	Static analysis	Quality (security, readability)	No
This work	Static + Execution	Methodology baseline	Pending

No published study measures static analysis impact on pass@k. This work provides the methodology and baseline to enable such evaluation.

2.0.4. 2.4 BENCHMARKS

HumanEval (Chen et al., 2021) contains 164 Python programming problems and is widely used for code generation evaluation. MBPP (Austin et al., 2021) contains 974 problems. Pass@k metrics measure functional correctness—the fraction of problems solved within k attempts. This study focuses on HumanEval, with MBPP extension as future work.

3. Methodology

3.0.1. 3.1 HYPOTHESIS STRUCTURE

The research question is formulated as a structured hypothesis with testable predictions:

Core Hypothesis (H-StaticFeedback-v1): Under iterative LLM code repair on HumanEval/MBPP, integrating static analyzer feedback (pylint/mypy) alongside execution feedback improves pass@k compared to execution-only feedback, because static analysis provides mechanistic error explanations.

Causal Mechanism:

1. Static analyzer produces warnings on LLM-generated code
2. LLM receives mechanistic feedback (why, not just where)
3. Targeted repairs lead to higher pass@k

Key Assumptions:

- A1: Static analyzers produce meaningful warnings on LLM code
- A2: LLMs can interpret static warnings into fixes
- A3: Static and execution feedback catch non-overlapping errors

- A4: HumanEval contains problems with static-detectable errors
- A5: Pylint severity filtering (disabling conventions) is effective

3.0.2. 3.2 GATE-BASED VALIDATION

A hierarchical gate structure is employed to validate assumptions before full experiments:

Gate	Type	Hypothesis	Criterion
h-e1	MUST_WORK	Existence	$\geq 30\%$ of problems have actionable warnings
h-m1	MUST_WORK	Mechanism	LLM interprets warnings into fixes
h-m2	SHOULD_WORK	Mechanism	Static/execution non-overlapping
h-c1	SHOULD_WORK	Comparison	Improvement stratified by warnings

MUST_WORK gates block downstream experiments if they fail. This design catches methodology issues early before resources are expended on invalid experiments.

3.0.3. 3.3 EXPERIMENTAL DESIGN

Variables:

- Independent: Feedback type (execution-only vs. static+execution)
- Dependent: pass@1, pass@5, pass@10
- Controlled: Base LLM, iteration budget (5), prompt format, static analyzer (pylint)

Dataset: HumanEval (164 problems)

Static Analysis Configuration:

```
PYLINT_ARGS = ["--output-format=json", "--disable=C,R"]
PYLINT_TIMEOUT_SEC = 30
ACTIONABLE_TYPES = ("error", "warning")
GATE_THRESHOLD = 0.30
```

Convention (C) and Refactoring (R) categories are disabled; only error and warning types are considered actionable.

3.0.4. 3.4 EXISTENCE GATE PROTOCOL

Before full pass@k evaluation, the existence assumption is validated: do static analyzers produce actionable warnings on benchmark code?

Protocol:

1. Load all HumanEval problems (164)
2. Extract code (canonical solutions or LLM generations)

3. Run pylint with configured filters

4. Count problems with at least one actionable warning

5. Gate passes if warning_rate $\geq 30\%$

Threshold rationale: Below 30%, fewer than 1 in 3 problems would receive static feedback, limiting both statistical power and practical utility. This threshold is conservative; lower rates might enable meaningful experiments but would require larger sample sizes.

3.0.5. 3.5 METRICS

- **Warning Rate:** Fraction of problems with ≥ 1 actionable warning
- **Warnings per Problem:** Average actionable warnings across all problems
- **Warning Distribution:** Breakdown by pylint warning code

4. Experimental Setup

4.0.1. 4.1 EXISTENCE GATE EXPERIMENT (H-E1)

The first causal mechanism step is tested: do static analyzers produce actionable warnings on benchmark code?

Configuration:

- Dataset: HumanEval (164 problems, full test set)
- Code source: Canonical solutions (proxy for LLM output due to API unavailability)
- Analyzer: pylint with --disable=C,R
- Gate threshold: $\geq 30\%$ of problems with ≥ 1 warning

Proxy rationale: Without OpenAI/Claude API access, canonical solutions were used as a lower-bound proxy. Canonical solutions are well-written human code; LLM-generated code typically exhibits more issues, so this represents a floor for expected warning rates.

4.0.2. 4.2 IMPLEMENTATION

Pipeline components:

- HumanEval loader: Loads problems from HuggingFace (openai_humaneval)

- Static analyzer: Runs pylint, parses JSON output, filters by severity
- Metrics aggregator: Computes warning counts and rates

Artifacts generated:

- `metrics.json`: Aggregated warning statistics
- `pylint\_results.json`: Per-problem pylint output
- `warning\_distribution.png`: Histogram of warnings per problem
- `warning\_type\_breakdown.png`: Distribution by warning category
- `gate\_metrics.png`: Threshold vs. actual comparison
- `top\_warning\_codes.png`: Most frequent warning codes

5. Results

5.0.1. 5.1 EXISTENCE GATE OUTCOME

Gate Result: FAIL

Metric	Value	Threshold
Warning Rate	9.15%	$\geq 30\%$
Problems with ≥ 1 Warning	15 / 164	—
Total Warnings	23	—
Average Warnings/Problem	0.14	—

The existence gate failed: only 9.15% of canonical solutions had actionable pylint warnings, well below the 30% threshold.

5.0.2. 5.2 WARNING DISTRIBUTION

Warning Code	Count	Description
bad-indentation	12	Incorrect indentation
unused-import	3	Imported but unused
bare-except	2	Catching all exceptions
unused-variable	1	Defined but unused
unnecessary-semicolon	1	Trailing semicolon
redefined-builtin	1	Shadowing builtin
pointless-string-statement	1	String with no effect
unreachable	1	Code after return
eval-used	1	Using eval()

The dominant warning type is **bad-indentation** (12 of 23 warnings, 52.2%). All 23 warnings fall into 9 distinct pylint codes.

5.0.3. 5.3 ANALYSIS

The low warning rate on canonical solutions reflects expected behavior. HumanEval canonical solutions are curated, high-quality reference implementations written by humans. They represent a floor, not a ceiling, for static warning rates.

The 9.15% rate does not invalidate the hypothesis—it invalidates the proxy. Canonical solutions are not LLM-generated code.

5.0.4. 5.4 GATE MECHANISM VALIDATION

The MUST_WORK gate correctly halted the pipeline:

- h-e1 FAIL blocked h-m1, h-m2, h-c1
- No downstream experiments were run on invalid proxy
- Methodology limitation was identified before full evaluation

[Figure placeholder: gate_metrics.png]

Figure 1. Gate Metrics Comparison

Figure 1: Gate threshold (30%) versus actual warning rate (9.15%) on canonical solutions.

[Figure placeholder: warning_distribution.png]

Figure 2. Warning Distribution

Figure 2: Distribution of warnings per problem across 164 HumanEval canonical solutions.

6. Discussion

6.0.1. 6.1 WHY THE PROXY FAILED

Canonical HumanEval solutions are carefully crafted reference implementations. They pass all tests by design and follow Python conventions. LLM-generated code—especially early attempts in a repair loop—typically contains more issues: undefined variables, type mismatches, incomplete logic.

Using canonical solutions as an LLM proxy conflates the reference answer with the model’s attempts. The 9.15% warning rate reflects human code quality, not LLM generation behavior.

6.0.2. 6.2 WHAT THE BASELINE ESTABLISHES

Despite the proxy limitation, the results provide value:

1. **Floor established:** Canonical solutions have approximately 9% warning rate. LLM code is expected to exceed this, providing room for the static analysis intervention to operate.

1. **Pipeline validated:** The pylint analysis infrastructure functions correctly. Components are reusable for future work with LLM API access.

1. **Warning types identified:** The 9 warning categories detected are actionable for code repair. They provide mechanistic explanations of why code is problematic.

6.0.3. 6.3 GATE DESIGN IMPLICATIONS

The MUST_WORK gate served its intended purpose. Without the existence check, the study would have:

- Expended compute on the wrong code source
- Produced results that do not test the hypothesis
- Potentially drawn incorrect conclusions

Gate-based validation catches methodology issues early.

6.0.4. 6.4 LIMITATIONS

Primary limitation: No LLM-generated code was tested. API access was unavailable during this study.

Secondary limitations:

- Single dataset (HumanEval only, not MBPP)
- Single static analyzer (pylint, not mypy)
- Single severity filter configuration

This is a methodology study establishing baseline and validating infrastructure—not a full hypothesis test.

6.0.5. 6.5 PATH FORWARD

To complete the hypothesis evaluation:

1. Re-run h-e1 with LLM API access (GPT-4 or Claude) and measure warning rates on generated code
2. If h-e1 passes (warning rate $\geq 30\%$), proceed to h-m1 (LLM interprets warnings) and h-m2 (non-overlapping errors)
3. Run full evaluation comparing pass@k between static+execution and execution-only conditions

The pipeline, baseline, and methodology are ready. The missing component is LLM API access.

7. Conclusion

This study asked whether static analyzers can improve LLM code repair. The answer remains open—but the methodology to find it is now established.

Findings: Canonical HumanEval solutions exhibit only 9.15% pylint warning rates (15 of 164 problems). This makes them unsuitable proxies for LLM-generated code. The MUST_WORK gate correctly identified this limitation before downstream experiments consumed resources.

Artifacts: A validated pylint analysis pipeline for code generation research, reusable components for HumanEval loading and static analysis, and a gate-based hypothesis validation framework.

Methodology insight: Gate-based validation functions as intended. By testing existence assumptions before full experiments, methodology issues are caught early.

Next steps: Re-run the existence gate with actual LLM generations. If warning rates exceed 30%, proceed to mechanism validation and full pass@k comparison.

Broader implication: Before evaluating any LLM intervention, validate that the intervention has signal. Canonical solutions are not LLM output—a lesson applicable beyond static analysis to any benchmark-based evaluation.

The question of whether static analysis improves pass@k merits investigation. This work provides the validated tools to pursue that investigation.

References

- Anonymous. The patchwork problem: Structural failures evade verification. *arXiv preprint arXiv:2607.08981*, 2026.
- Austin, J. et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.
- Blyth, H. et al. Static analysis as a feedback loop for LLM code generation. *arXiv preprint arXiv:2508.14419*, 2025.
- Chen, H. et al. Helping LLMs improve code generation with feedback from testing and static analysis. *arXiv preprint arXiv:2412.14841*, 2024.
- Chen, M. et al. Evaluating large language models

trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

Chen, X., Lin, M., Schärli, N., and Zhou, D. Teaching large language models to self-debug. *arXiv preprint arXiv:2304.05128*, 2023.

Madaan, A. et al. Self-refine: Iterative refinement with self-feedback. *NeurIPS*, 2023.

Pylint Development Team. Pylint: Python static code analyzer. <https://github.com/pylint-dev/pylint>, 2024.

Zhong, L. et al. LDB: A large language model debugger via verifying runtime execution step-by-step. *arXiv preprint arXiv:2402.16906*, 2024.